GRADIENT BOOSTING

Height (m)	Fav Color	Gender	Weight (kg)
1.6	Blue	M	88
1.6	Green	F	76
1.5	Blue	F	56
1.8	Red	M	73
1.5	Green	M	77
1.4	Blue	F	57

↳ when predicting a continuous value: GB for regression

Adaboost: builds a short stump on the training data and the amount of say that the stump has on the final output is based on how well it compensated for previous errors

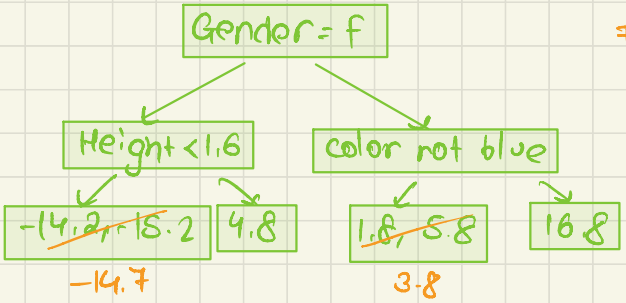
Gradient Boosting: makes a single leaf with initial guesses of weights of all samples
- when trying to predict a continuous value like weight, 1st guess is the average value
- then, builds a tree, based on the errors made by the previous tree, and size is fixed
- tree > stump

↳ max_leaves = blw 8 and 32 in practical scenarios
↳ scales all the trees by the same amount

When does it end?
- Made # trees we asked for
- Additional trees fail to improve the fit

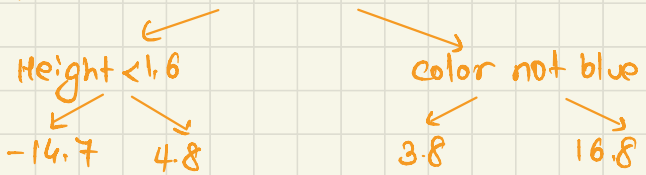
① Calculate avg. weight = 71.2
② Build a tree based on errors made by the previous tree to predict the residuals

Height (m)	Fav Color	Gender	Weight (kg)	Preudo-Residual
1.6	Blue	M	88	16.8
1.6	Green	F	76	4.8
1.5	Blue	F	56	-15.2
1.8	Red	M	73	1.8
1.5	Green	M	77	5.8
1.4	Blue	F	57	-14.2



leaves (4) < # residuals (6)
∴ grouped them
take avg.

combine: 71.2 (avg. wt.) + Gender = F



↳ 71.2 + 16.8 = 88.0
fits exactly ∴ low bias
and high variance
↓
use learning rate $\in (0,1)$

With $\alpha = 0.1$, $71.2 + 0.1(16.8) = 72.9$ ∈ not as good as actual but better than original leaf of 71.2

:
repeat

Height (m)	Fav Color	Gender	Weight (kg)	Pseudo residual
1.6	Blue	M	88	88 - 72.9 = 15.1
1.6	Green	F	76	4.3
1.5	Blue	F	56	-13.7
1.8	Red	M	73	1.4
1.5	Green	M	77	5.4
1.4	Blue	F	57	-12.7

all better than earlier pseudo-residuals ∴ taken step in right direction

Do the same thing over and over...

sample calculation: $71.2 + (0.1 \times 16.8) + (0.1 \times 15.1)$
$= 74.4$

i.e. another small step

Each time we add a new tree, we ↓ the residuals

Pros:
- captures complex patterns in the data
- flexible ∴ can do classification and regression

Cons:
- computationally expensive if dataset is large or trees are deep
- can be prone to overfitting
- less interpretable if complex structure

eg:

Height (m)	Fav Color	Gender	Weight
1.6	Blue	M	88
1.6	Green	F	76
1.5	Blue	F	56

$\text{Data} = \{(x_i, y_i)\}_{i=1}^{n}$

Differentiable Loss Fx^n = $\frac{1}{2}$ (observed − predicted)^2

↓

why? even if we don't have it, the magnitudes of liners would be relative

Reason: $\frac{d}{d\,\text{Predicted}} \frac{1}{2}(\text{obs} - \text{pred})^2$

↓ chain rule

$\frac{2}{2}(\text{obs} - \text{pred})(-1)$

$= -(\text{obs} - \text{pred})$

① Initialize model with constant value : $f_0(x) = \underset{\gamma}{\text{argmin}} \sum_{i=1}^{n} L(y_i, \gamma)$ (obs, pred)

↓

need to find a pred γ that minimizes sum

② for $m = 1$ to M: making M trees; generally, $M = 100$

→ compute $r_{im} = -\left(\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right)_{F(x) = f_{m-1}(x)}$ residual

derivative of loss fxn relative to predicted value

→ fit a regression tree to the r_{im} values and create terminal regions R_{jm} for $j = 1, \ldots, J_m$

build a regression tree to predict weights

→ for $j = 1 \ldots J_m$, compute

$$\gamma_{jm} = \underset{\gamma}{\operatorname{argmin}} \sum_{x_i \in R_{ij}} L(y_i, f_{m-1}(x_i) + \gamma)$$

→ update $f_m(x) = f_{m-1}(x) + \nu \sum_{j=1}^{J_m} \gamma_m \mathbb{I}(x \in R_{jm})$